

DDA Line Drawing Algorithm

Lab Report: Implementation of DDA Line Drawing Algorithm in Python

1. Introduction

In computer graphics, line drawing algorithms are fundamental for rendering lines on a raster display. The Digital Differential Analyzer (DDA) algorithm is one of the simplest methods used to interpolate values over an interval between two end points. It is used to draw straight lines by calculating intermediate positions based on the line's slope.

2. Objective

To understand and implement the DDA (Digital Differential Analyzer) algorithm in Python and analyze its functioning in line drawing.

3. Algorithm Explanation

The DDA algorithm works by calculating either x or y depending on the line's slope and incrementally plotting the intermediate points between the start and end coordinates.

Key Principles:

- If $|m| \leq 1$, we increment x by 1 and compute the next y using:

$$y_{i+1} = y_i + m$$

- If $|m| > 1$, we increment y by 1 and compute the next x using:

$$x_{i+1} = x_i + 1/m$$

- This continues until x reaches x (for $|m| \leq 1$) or y reaches y (for $|m| > 1$).

- All calculated values are rounded to the nearest pixel.

Why use it?

- The DDA algorithm avoids multiplication-heavy equations by using addition.

- Floating-point operations are still used but are efficient.
- It may accumulate small rounding errors for long lines due to floating-point precision limits.

Clarifying the Direction Logic:

- For upward or rightward lines: we increment by +1.
- For downward or leftward lines: we increment by -1 (or swap endpoints to keep increment +1).

4. DDA Algorithm Steps

1. Input the starting point (x_1, y_1) and ending point (x_2, y_2) .
2. Calculate $dx = x_2 - x_1$ and $dy = y_2 - y_1$.
3. Calculate slope $m = dy / dx$ (handle $dx = 0$ case separately).
4. If $|m| \leq 1$:
 - Increment x by 1 each step.
 - Compute $y = y + m$ each time.
 - Stop when $x > x_2$.
5. Else:
 - Increment y by 1 each step.
 - Compute $x = x + 1/m$ each time.
 - Stop when $y > y_2$.
6. Round and plot each (x, y) value as a pixel.

5. Pseudocode

function DDA(x_1, y_1, x_2, y_2):

$dx = x_2 - x_1$

$dy = y_2 - y_1$

```

if dx == 0:
    m = infinity
else:
    m = dy / dx

if |m| <= 1:
    if x1 > x2:
        swap x1, y1 with x2, y2
    x = x1
    y = y1
    while x <= x2:
        plot(round(x), round(y))
        x = x + 1
        y = y + m
    else:
        if y1 > y2:
            swap x1, y1 with x2, y2
        x = x1
        y = y1
        while y <= y2:
            plot(round(x), round(y))
            y = y + 1
            x = x + 1/m

```

6. Sample Python Program (Simplified Textbook Version)

```
import matplotlib.pyplot as plt
```

```

def dda_line(x1, y1, x2, y2):

    dx = x2 - x1

    dy = y2 - y1


    # Handle vertical line to avoid division by zero

    if dx == 0:

        m = float('inf') # infinite slope

    else:

        m = dy / dx


    # Prepare lists to store points

    x_points = []
    y_points = []


    # Case 1: |slope| <= 1, move in x-direction

    if abs(m) <= 1:

        if x1 > x2:

            x1, y1, x2, y2 = x2, y2, x1, y1 # swap to go left to right

            x = x1

            y = y1

            while x <= x2:

                x_points.append(round(x))

                y_points.append(round(y))

                x += 1

                y += m # y = y + m

```

```

# Case 2: |slope| > 1, move in y-direction
else:
    if y1 > y2:
        x1, y1, x2, y2 = x2, y2, x1, y1 # swap to go bottom to top
    x = x1
    y = y1
    while y <= y2:
        x_points.append(round(x))
        y_points.append(round(y))
        y += 1
        x += 1 / m # x = x + (1/m)

# Plotting the points
plt.plot(x_points, y_points, marker='o', color='blue')
plt.title('Simplified DDA Line Drawing Algorithm')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid(True)
plt.show()

```

Sample run

```
dda_line(2, 3, 12, 8)
```

7. Tasks to Perform in Lab

1. Take user input for starting and ending points and draw a line using DDA.

2. Test the program with different types of lines:

- Horizontal
- Vertical
- Diagonal
- Steep and shallow slopes

3. Add options to change the color or thickness of the line.

8. Practice Exercises (Home Tasks)

1. Modify the DDA program to draw a triangle by connecting three vertices..
2. Plot points using integer-only rounding logic (no `round()` function) and observe differences.
3. Draw a grid and visualize line drawing over it using matplotlib.

9. Conclusion

The DDA line drawing algorithm is a stepping stone in understanding how graphics rendering works at the pixel level. It builds the foundation for more complex rendering algorithms and helps grasp key computational geometry concepts in graphics programming. The simplified Python implementation closely follows the textbook approach and is a great tool for hands-on learning.

10. References

- Computer Graphics by Donald Hearn and M. Pauline Baker
- [https://en.wikipedia.org/wiki/Digital_differential_analyzer_\(graphics_algorithm\)](https://en.wikipedia.org/wiki/Digital_differential_analyzer_(graphics_algorithm))
- https://www.tutorialspoint.com/computer_graphics/line_generation_algorithm.htm